

CALS Attendance Ledger: 근태 Connector를 원장으로 확장하기

근태 연동은 API 연결 문제가 아니라, 조직의 근무 기록을 보존하는 원장 설계 문제다.

요약

CALS는 단순 출퇴근 connector가 아니라 ERP·ESS·HR 계산이 참조할 수 있는 근태 원장으로 설계했다. 외부 단말기나 시스템에서 들어오는 출퇴근 이벤트를 그대로 덮어쓰지 않고, 사건의 누적, 보정, 승인, 재처리, audit을 기준으로 관리했다.

핵심 판단은 “최신 상태를 저장하는 테이블”이 아니라 “왜 그런 상태가 되었는지 설명할 수 있는 원장”이 필요하다는 점이었다.

문제 정의

근태 데이터는 상태처럼 보이지만 실제로는 사건의 누적이다. 출근, 퇴근, 외근, 지각, 조퇴, 보정 요청, 관리자 승인, 연차 반영, 야근 계산, 휴일 근무가 시간 순서로 쌓인다. 최종 상태만 저장하면 계산은 빠르지만, 분쟁과 정정 요청이 발생했을 때 설명할 수 없다.

connector 중심 구조에는 한계가 있었다.

- 외부 단말기 지연 전송과 중복 전송을 처리해야 한다.
- 같은 직원의 출퇴근 이벤트가 순서와 다르게 들어올 수 있다.
- 조직별 근무 규칙과 예외 근무가 다르다.
- 보정 요청과 승인 이력이 계산 결과와 함께 남아야 한다.
- ERP와 ESS가 동일한 기준 데이터를 참조해야 한다.

책임 경계

계층	책임
Connector	외부 시스템 이벤트를 수신하고 표준 이벤트로 변환한다.
Ledger	원본 이벤트, 보정, 승인, 계산 기준을 보존한다.

계층	책임
Rule Engine	조직별 근무 규칙을 적용해 계산 결과를 만든다.
ERP/ESS	원장을 기준으로 조회하고 업무 화면에 반영한다.
Audit	누가 어떤 근태 기록을 왜 보정했는지 남긴다.

핵심 설계 결정

최신 상태보다 사건의 누적을 우선했다

출퇴근 기록은 단순히 IN, OUT 상태로 끝나지 않는다. 나중에 보정되거나 승인될 수 있고, 계산 기준이 변경될 수 있다. 따라서 원본 이벤트를 유지하고, 보정과 승인도 별도 사건으로 남기는 구조가 필요했다.

```
raw_event
-> normalized_event
  -> ledger_entry
    -> adjustment_request
      -> approval
        -> calculated_attendance
```

이 구조는 저장량이 늘어나지만, 근태 분쟁과 재계산 요구에 대응할 수 있다.

중복과 지연을 예외로 보지 않았다

외부 단말기와 연동 시스템에서는 중복, 지연, 순서 역전이 발생한다. 이를 예외 로그로만 처리하면 운영자가 수동으로 보정해야 한다. CALS에서는 idempotency key, event time, received time, source system, employee mapping을 기준으로 중복과 지연을 정상 처리 흐름 안에 넣었다.

ERP와 ESS를 직접 흔들지 않았다

외부 이벤트가 들어올 때마다 ERP와 ESS 상태를 직접 변경하면 장애 전파 범위가 커진다. CALS는 원장을 기준으로 계산 결과를 만들고, ERP/ESS는 검증된 결과를 참조하도록 분리했다.

버린 선택과 이유

근태 connector를 단순 API 연동으로 처리하는 방식은 제외했다. 출퇴근 데이터는 외부 장비, 모바일, 관리자 보정, 연차, 휴일 규칙, HR 마감과 연결된다. 최신값만 동기화하면 사후 보정과 계산 재현이 어렵다.

ERP와 ESS가 외부 근태 시스템을 직접 호출하는 방식도 피했다. 외부 연동 장애가 내부 업무 시스템의 응답 지연으로 이어지고, 동일 데이터를 여러 시스템이 서로 다른 기준으로 해석할 수 있기 때문이다.

중복 이벤트를 단순 오류로 버리는 방식도 위험했다. 같은 출근 기록이 다시 들어오는 것은 외부 시스템 재전송, 네트워크 재시도, 장비 동기화 지연에서 자연스럽게 발생할 수 있다. 중복은 무시 대상이 아니라 idempotency 기준으로 해석해야 했다.

근태 원장에서 중요하게 본 기록

근태 데이터는 현재 상태보다 변경 이력과 계산 근거가 중요하다. 운영자가 확인해야 하는 기록은 다음과 같았다.

기록	목적
원본 event id	외부 시스템에서 들어온 사건을 추적한다.
보정 전/후 값	관리자 수정과 자동 계산 결과를 분리한다.
적용된 근무 규칙 version	당시 어떤 규칙으로 지각·연장·휴일을 계산했는지 재현한다.
HR 마감 상태	이미 마감된 기간의 변경이 급여와 연차에 미치는 영향을 통제한다.
재계산 이력	사후 보정 이후 어떤 결과가 바뀌었는지 설명한다.

운영 검증

검증 기준은 “계산 결과가 맞는가”만이 아니었다. 왜 그런 결과가 나왔는지, 어떤 원본 이벤트와 보정 승인에 의해 계산되었는지 설명할 수 있어야 했다.

주요 검증 항목은 다음과 같았다.

- 중복 이벤트가 들어와도 원장이 오염되지 않는가.
- 지연 이벤트가 나중에 들어왔을 때 재계산 범위가 제한되는가.
- 관리자가 보정한 기록과 승인 이력이 audit으로 남는가.
- 조직별 근무 규칙이 코드 변경 없이 확장 가능한가.
- ERP/ESS 조회 결과가 원장과 불일치하지 않는가.

운영 결과

근태 데이터의 기준이 connector별 최신 상태에서 원장 중심 구조로 이동했다. 운영자는 단순 결과뿐 아니라 원본 이벤트, 보정 이력, 승인 이력, 계산 근거를 함께 확인하게 되었다. 외부 연동 장애가 발생해도 ERP와 ESS를 직접 흔들지 않고 재처리 가능한 단위로 복구했다.

역할과 책임

근태 connector 구조의 한계 분석, 원장 모델, event normalization, 보정·승인·audit 흐름, ERP/ESS 연동 경계, 재처리 기준을 설계했다.

설계 결론

근태 시스템의 핵심은 출퇴근 API 연동이 아니라 조직의 근무 기록을 신뢰 가능한 원장으로 남기는 것이다. 최신 상태만 저장하는 구조는 빠르지만 설명력이 부족하다. CALS는 근태 데이터를 계산 결과가 아니라 증거와 이력이 있는 운영 자산으로 다루기 위한 구조였다.